

Before formal testing, we should prepare a stable version of EcoTrack, repeatable test data, and clear pass/fail criteria. We can continue developer checks throughout; we don't need to wait until everything below is finished.

I checked the current configuration. Two concrete preparation gaps are: **CI has no Redis service**, and **EAS only has a development build profile**.

Do these tasks in this order:

1. **Finalize the features included in this release.**

Compare the SRS with the current system: authentication, profiles, organization approval, memberships, incident reporting/review, maps, cleanup events, participation, completion evidence, achievements, dashboards, notifications, and Redis behavior. Mark each requirement as implemented, incomplete, or explicitly deferred. This prevents testing against outdated requirements.

2. **Define expected results and acceptance criteria.**

Write down the permitted actions for citizens, organization members, coordinators, organization admins, and Super Admin. Define notification recipients, incident/event transitions, and service-area visibility. For performance, agree on response-time targets, concurrent users, dataset size, and acceptable error rates **before measuring results**.

3. **Walk through the complete workflows and fix blockers.**

Perform a quick developer walkthrough covering signup → report incident → organization review → publish event → join → record completion → rewards/notifications. Also cover organization approval and membership management. Recheck the earlier status-change error, login/CORS problems, notification permissions, and notifications after logout; don't assume those are resolved without verification.

4. **Make the database setup reproducible.**

Confirm that a fresh test database can apply all migrations, initialize PostGIS, and load required reference data such as incident categories and service-area boundaries. Check that the deployment database has the same migration history. Prepare controlled test-data cleanup without resetting existing user data.

5. **Prepare isolated test accounts and datasets.**

Create Organization A and Organization B, separate admins/members, assigned and unassigned coordinators, ordinary citizens, and a Super Admin. Include overlapping and non-overlapping service areas, incidents in different states,

full/cancelled/completed events, and users with multiple memberships. These give us repeatable functional and tenant-isolation tests.

6. Run the existing automated checks and fix failures.

Run backend tests/typecheck/build, web tests/lint/build, and mobile tests/typecheck/Expo Doctor/bundling. Record skipped tests separately from passes. This establishes a baseline before adding course-required testing evidence.

7. Complete CI coverage for Redis.

Add a Redis service and the required test environment variables to [ci.yml](C:/campus/projects/Eco-Track/.github/workflows/ci.yml). The Redis integration test currently skips without TEST_REDIS_URL. Ensure automated checks actually exercise caching, invalidation, shared rate limits, and worker monitoring.

8. Prepare a staging environment resembling deployment.

Run the web application, API, notification worker, PostgreSQL/PostGIS, and Redis together. Use HTTPS, correct CORS origins, Supabase callback URLs, storage configuration, and environment variables. Keep test storage and notification recipients isolated. **Use a separate Redis instance for staging:** the current cache prefix does not isolate BullMQ's queue keys.

9. Create an installable testing APK.

Add a standalone preview profile to [eas.json](C:/campus/projects/Eco-Track/mobile/eas.json), configure the staging API address, and verify installation, magic-link callbacks, permissions, and push registration. Testers should be able to open the app without Metro or your laptop. Use a release-mode build for performance measurements because development mode adds overhead. [React Native guidance](#)

10. Prepare the test cases and evidence templates.

Each case should contain: requirement ID, role, prerequisites, input, steps, expected result, actual result, status, build/version, and evidence. Include denied permissions, expired sessions, failed uploads, duplicate submissions, cross-organization access, Redis outages, and worker restarts. Prepare a defect log with severity and retest status.

11. Set up the selected testing tools and measurement conditions.

Your course lists examples; we don't need every tool. Prepare API requests, web/mobile automation scenarios, load-test scripts, accessibility checks, and SUS tasks/questionnaires. For load testing, define realistic user activity and collect response times, errors, CPU, memory, and queue growth; run JMeter load tests in

CLI mode. For mobile resource testing, fix the device, network, brightness, and task duration so comparisons are meaningful. [JMeter guidance](#)

12. Identify and freeze the version entering formal testing.

Record the source revision, APK build, deployed configuration, and database migration version. Confirm the main workflows work, test accounts are ready, logs are accessible, and no known blocker prevents testing. Subsequent fixes need tracked retests and relevant regression checks.

Your part: confirm the final scope and acceptance targets, provide test email accounts/devices, arrange usability participants, and decide the staging hosting/budget. I can handle the repository audit, configuration, repeatable fixtures, CI changes, and test preparation.

Production deployment comes **after** formal testing, defect fixes, regression checks, and a verified backup/restore and rollback procedure.